
EMERITUS
Learn. From the world's best.

Deep Feedforward Neural Networks - II

EMERITUS
Learn. From the world's best.

Implementing Regularization & Normalization in Keras

Session Goals

Improve Model Stability and Performance

Dropout & Dropconnect in Keras
Apply these regularisation techniques to prevent overfitting

Input Normalisation
Stabilise inputs for improved training performance

Batch Normalisation (Pre vs Post Activation)
Analyse how placement affects network behaviour

Layer Normalisation in RNNs/Transformers
Implement sequence-specific normalisation

Performance Evaluation
Assess impact on model accuracy and training stability

Dropout in Keras

Prevent Overfitting with Random Neuron Drop

Dropout is a regularisation technique that prevents overfitting by randomly disabling a fraction of neurons during training

```
from keras.models import Sequential
from keras.layers import Dense, Dropout
model = Sequential([    Dense(128, activation='relu', input_shape=
(784,)),    Dropout(0.5),    Dense(10, activation='softmax')])
```

Dropout(0.5) switches off 50% of neurons per step, making the network more resilient

DropConnect (Custom Layer)

Regularise by Dropping Connections

Dropout zeros outputs, while DropConnect zeros weights. Keras has no built-in DropConnect, but we can implement it:

```
import tensorflow as tf
from keras.layers import Dense
class DropConnect(Dense):
        def call(self, inputs, training=False):
         w = self.kernel
          if training:
                w = tf.nn.dropout(w, rate=0.3)
                        # drop connections
                        return tf.matmul(inputs, w) + self.bias
```

Drops 30% of connections during training, providing an alternative regularisation method

Input Normalisation

Stabilise Training with Standardised Inputs

Input normalisation stabilises training by standardising feature scales. Keras provides a dedicated layer for this purpose.

```
from keras.layers import Normalization
norm = Normalization()
norm.adapt(train_data)
model = Sequential([norm,    Dense(64, activation='relu'),    Dense(1)])
```

adapt() calculates mean and variance from training data for normalisation during training and inference

Batch Normalisation

Placement Effects on Training

Pre-activation Batch Normalization

```
model.add(Dense(128))
model.add(BatchNormalization())
model.add(Activation('relu'))
```

Normalises outputs before activation, improving gradient flow and often speeding up convergence

Post-activation Batch Normalization

```
model.add(Dense(128, activation='relu'))
model.add(BatchNormalization())
```

Normalises after activation; useful in some architectures but less common

Layer Normalisation

Feature-wise Normalisation for Sequence Models

Layer normalisation standardises features, independent of batch size, making it particularly useful for:

- Recurrent Neural Networks (RNNs)
- Transformer architectures
- Situations with small or variable batch sizes

```
from keras.layers import LayerNormalization
x = Dense(64)(inputs)
x = LayerNormalization()(x)
```

Comparative Analysis

Regularisation & Normalisation Comparison

Method	Helps With	Example Use
Dropout	Overfitting	CNNs, DNNs
DropConnect	Sparse connections	Custom models
Batch Norm	Faster convergence	CNNs
Layer Norm	Sequence models	RNNs, NLP

Each technique targets specific neural network training challenges, chosen based on model, data, and goals.

Regularisation (Dropout, DropConnect) reduces overfitting, while normalisation (Batch Norm, Layer Norm) improves stability and convergence speed

Hands-On Activity

Experiment with Regularisation & Normalisation

- **Build CNN on CIFAR-10**
  Train a basic CNN for image classification.
- **Add Dropout + BatchNorm**
  Insert Dropout after Dense layers and Batch Normalisation after convolutions.
- **Create DropConnect + LayerNorm version**
  Use a custom DropConnect layer and Layer Normalisation.
- **Compare Results**
  Examine accuracy and training curves to see the effects of each approach.

Summary

Key Takeaways

- **Regularisation** (Dropout, DropConnect) prevents overfitting by adding controlled noise during training
- **Normalisation** (Input Norm, Batch Norm, Layer Norm) stabilises and speeds up training by controlling layer input distribution
- Technique choice depends on model, dataset, and challenges
- Combining techniques often works best but needs careful tuning

Q&A Session

- Open floor for questions and clarifications.